URMACK · FLAGSHIP PRODUCT · CONFIDENTIAL

Urmack Watch

Continuous economic-invariant monitoring for DeFi protocols. Clients declare invariants once; Urmack Watch evaluates them against on-chain state every block-interval and alerts, with evidence, the moment one breaks.

Document 01 — System Architecture
Companion documents 02 — Flows & SDLC Roadmap · 03 — Product Designs
Version 1.0 · 16 October 2026
Author Office of the Interim CTO, Urmack
Status Approved for build — Phase 0 begins on signature

1. Product definition

Problem. Audits are point-in-time. Between engagements, protocols drift: an upgrade shifts accounting, an oracle degrades, a privileged role is abused, liquidity hollows out. Existing monitoring (Forta-style node alerting, Tenderly alerts) watches *transactions and log patterns*; it does not evaluate whether the protocol's economic rules still hold.

Product. A multi-tenant SaaS where a protocol team declares *invariants* — properties that must always be true (solvency, share-price bounds, collateral ratios, conservation of funds) — and Urmack Watch continuously evaluates them against live chain state. A breach produces a severity-classified alert with a full evidence pack, not a raw log line.

Buyer. Protocol founders/CTOs, DAO security working groups, and (Tier 3) funds buying portfolio coverage.

Mooat. Urmack's research practice supplies the invariant library and the economic reasoning behind severity classification — the part general-purpose monitoring cannot commoditize.

2. SDLC selection

Three candidate methodologies were evaluated for a flagship SaaS built by a small team:

METHOD	FIT ASSESSMENT	VERDICT
Waterfall	Requirements for a monitoring product cannot be fully known before live-protocol feedback; long feedback cycles are fatal for a first-of-kind economic monitoring engine.	Rejected
Pure Scrum	Good cadence, but Sprint boundaries alone do not force early confrontation of the highest-risk unknowns (RPC reliability, invariant expressiveness).	Rejected as primary
Iterative-incremental, risk-driven (Agile delivery cadence + explicit architecture-risk spikes per phase)	Delivers a walking skeleton early, forces the riskiest engineering (chain adapters, spec language, evaluation engine) through time-boxed spikes *before* feature investment, and keeps each increment demoable to a design partner.	**Selected**

> **Selected SDLC — Iterative & Incremental with risk-driven spikes.** Four phases, each ending in a demoable release and a hard gate. spikes are time-boxed (≤2 weeks) engineering experiments with explicit pass/fail criteria; a failed spike re-plans the phase rather than being absorbed silently.

PHASE	GOAL	EXIT GATE
0 — Spike	Kill the top-3 technical risks	All spikes pass criteria or phase is re-planned; walking skeleton runs one invariant on one chain
1 — Alpha	Real monitoring for one design partner	30 consecutive days of correct evaluations; alert delivered < 60 s after induced breach
2 — Beta	Self-serve onboarding, 5–10 protocols	Onboarding < 30 min without Urmack involvement; billing live; RLS pen-test passed
3 — GA	Portfolio tier, multi-chain, public launch	99.9% evaluation availability over 30 days; IR escalation integration

3. System architecture

Seven bounded contexts. Every component is independently deployable; the evaluation plane (3–6) is isolated from the control plane (1–2, 7) so portal traffic can never starve monitoring.

1 · Control API `CONTROL PLANE`

REST/JSON API for orgs, users, protocols, contracts, invariant specs and notification routes. Supabase Auth (magic link → later SSO) + Postgres with row-level security as the hard multi-tenant boundary.

2 · Onboarding & Spec Compiler `CONTROL PLANE`

Validates addresses/ABIs, resolves proxy implementations, compiles the human-readable invariant spec (YAML) into an *evaluation plan*: the exact `eth_call` batch, storage reads, event queries and the assertion function. Runs a *dry-run backfill* over N historical days to prove the invariant before arming.

3 · Scheduler `EVALUATION PLANE`

Block-height-driven tick service (per chain). Emits evaluation jobs per protocol/plan at the subscription cadence (Tier 1: 5-min · Tier 2: per-block). Guarantees exactly-one-in-flight per plan via Redis locks; missed ticks feed the heartbeat (§7).

4 · Chain Data Layer `EVALUATION PLANE`

Multi-provider RPC pool (Alchemy/QuickNode + fallback) with per-chain adapters, confirmation-depth handling (default 2 blocks L1 / 1 block L2), reorg detection, and a cached ABI/implementation registry. Read-only keys only — the product never holds signing material.

5 · Snapshot & Evaluation Engine `EVALUATION PLANE`

Builds a consistent state snapshot per tick (batched `eth_call` + event window), then executes the compiled assertions: point-in-time, temporal ("X holds for N consecutive blocks"), and derived-metric checks (ratios, share-price math). Emits `EvaluationResult{plan, block, values, pass, margin}`.

6 · Detection, Severity & Alert Router EVALUATION PLANE

Classifies breaches (Critical / Warning / Anomaly per §6), applies dedup + cooldown (no alert storms), packages the evidence pack (§8), and routes to channels: webhook (HMAC-signed), Telegram, Discord, Slack, PagerDuty, email — WhatsApp via Cloud API on the premium tier.

7 · Client Portal CONTROL PLANE

Next.js app: protocol health dashboard, invariant editor with live dry-run, alert history with evidence viewer, routing config, team management. Read/write only through the Control API.

Supporting data services

- **Postgres (Supabase, RLS)** — tenancy, specs, plans, alert records, audit log.
- **TimescaleDB hypertables** — every `EvaluationResult` (pass or fail) as time-series: powers margin charts ("how close did we come?") and post-incident timelines.
- **Redis** — tick locks, provider circuit breakers, hot ABI cache.
- **Object store (S3/R2)** — evidence packs (immutable, 90-day retention; exportable for IR retainers).

4. Data model (core entities)

ENTITY	KEY FIELDS	NOTES
`org`	id, name, plan_tier, billing_status	Tenancy root; every row below carries `org_id` for RLS
`protocol`	id, org_id, name, category, primary_chain	One monitored protocol
`contract`	id, protocol_id, chain, address, abi_ref, proxy_impl	Implementation-change watcher attached here
`invariant_spec`	id, protocol_id, yaml_source, kind, severity, cadence, status	Source of truth; compiled plans reference it
`eval_plan`	id, spec_id, compiled_json, version, checksum	Immutable per version — audits what ran
`evaluation`	plan_id, block, ts, values, pass, margin	Timescale hypertable; never deleted
`alert`	id, spec_id, severity, block, status, evidence_uri	status: open → acked → resolved / false-positive
`route`	id, org_id, channel, config, severity_filter	Secrets stored encrypted; webhook HMAC key per route
`heartbeat`	protocol_id, last_eval_ts, expected_interval	Drives dead-man monitoring of the monitor

5. Technology stack

LAYER	CHOICE	RATIONALE
Portal	Next.js + TypeScript (strict) + Tailwind	Master-prompt house stack; RSC keeps client JS small
API	Hono on Node (or Cloudflare Workers)	Fetch-standard, typed, deployable anywhere
Evaluation workers	TypeScript + viem, BullMQ on Redis	viem's typed ABIs; queue gives retries/backpressure
Spec language	YAML → JSON-Schema → compiled plan (Zod-validated)	Human-readable, formally checkable, versionable
Storage	Supabase Postgres + TimescaleDB, S3/R2 evidence	Existing Supabase relationship; RLS = tenancy for free
Alerting	Webhooks (HMAC), Telegram, Discord, Slack, PagerDuty, WhatsApp Cloud API	Buyer's existing on-call workflows
Observability	OpenTelemetry → Grafana Cloud; dead-man alerts on the scheduler itself	A monitor that fails silently is worse than none

6. Severity model

SEVERITY	DEFINITION	DEFAULT RESPONSE
Critical	Solvency, conservation or access-control invariant broken at finality depth. Funds are (or may be) at risk *now*.	All routes + PagerDuty/WhatsApp immediately; IR playbook attach
Warning	Margin erosion: invariant still holds but crossed a configured safety band (e.g. collateral ratio within 5% of limit).	Business-hours routes; digest if repeated
Anomaly	Statistical deviation from the spec's own 30-day baseline (share price jump, TVL outflow).	Digest; no paging

7. Failure modes & mitigations

FAILURE	MITIGATION
RPC provider outage	Multi-provider pool with circuit breakers; degrade cadence rather than skip; if no evaluation within 3× expected interval, **heartbeat dead-man alert** — the monitor reports its own blindness.
Chain reorg	Finality depth per chain (configurable); breach alerts wait for depth, sub-depth violations recorded as pending.
Spec author error	Dry-run backfill (§2) before arming; plans are versioned and immutable; changing a spec never rewrites history.
False-positive storm	Dedup + cooldown + per-route rate caps; anomaly channel never pages.
Tenant data leak	Postgres RLS on every table; evidence objects signed-URL only; quarterly access review; RLS pen-test is a Beta gate.
Insider compromise of Urmack	No signing keys ever stored; read-only RPC credentials scoped per provider; alert routing changes require org-owner re-auth and are audit-logged.

8. Evidence pack (the product's differentiator)

Every Critical/Warning alert ships a signed, self-contained bundle: (1) breached invariant in human language + raw spec, (2) evaluated values vs. thresholds at breach block, (3) 24 h margin time-series, (4) the triggering transaction(s) with decoded traces, (5) recent privileged events on in-scope contracts, (6) Uremack first-checks playbook. This is the artifact that turns an alert into an engagement and feeds the IR retainer upsell.

9. Scaling path

v1: single evaluation worker pool, protocol-level partition keys. v2: per-chain partitions, snapshot caching across protocols sharing an RPC provider, spec-level precomputation. v3: portfolio tier — one evaluation pass amortized across funds' overlapping protocol sets. Timeseries retention: full resolution 90 d, hourly rollup 2 y (hypertable policies).

10. Explicit non-goals (v1)

- No transaction simulation or auto-response (no signing material — ever).
- No off-chain data sources (CEX prices, team comms) — on-chain state only, stated plainly in docs.
- No claim of complete exploit detection. The docs will say what the product does and does not see, matching Urmack's disclosure principles.